Python String Operations

Lecture Notes - Part 2

String Basics

Creating Strings

```
s = "HELLO WORLD"
print(s)
print(type(s))
```

```
HELLO WORLD
<class 'str'>
```

String Indexing

Python supports both positive and negative indexing:

```
print(s[0])
print(s[-len(s)])
print(s[len(s)-1])
print(s[-1])
```

```
H
H
D
D
```

> ⚠️ *Strings are **immutable** objects!*

```
s[0] = "M"
```

```
TypeError: 'str' object does not support item assignment
```

String Slicing

*Syntax: **s[start:stop:step]***

- *Default start = 0*
- *Default stop = len(s)*
- *Default step = 1*

Basic Slicing Examples

```
s = "HELLO WORLD"
print(s[0:5:1])
print(s[:5])
print(s[3:10])
print(s[0:len(s):1])
print(s[::])
print(s[5:len(s):1])
print(s[5:])
```

```
HELLO
HELLO
LO WORL
HELLO WORLD
HELLO WORLD
WORLD
WORLD
```

Advanced Slicing

```
# Last 5 characters
print(s[-5:])

# First 4 characters
print(s[:4])

s1 = "PYTHON PROGRAMMING"
# Extract PYTHON
print(s1[:6])
print(s1[-18:-12])

# Extract PROGRAM
print(s1[7:14])
print(s1[-11:-4])

# Extract GRAM
print(s1[10:14])
print(s1[-8:-4])
```

```
WORLD
HELL
PYTHON
PYTHON
PROGRAM
PROGRAM
GRAM
GRAM
```

String Reversal Techniques

Reversing Strings

```
s1 = "PYTHON PROGRAMMING"

# Extract GNIMMARGORP (reverse of "PROGRAMMING")
print(s1[:6:-1])
print(s1[:len(s1)-6:-1])

# Reverse only "PYTHON" portion
print(s1[5::-1])
print(s1[-13::-1])

# Last 5 characters
print(s1[-5:])

# First 3 characters in reverse order
print(s1[2::-1])

# Reverse the whole string
print(s1[::-1])
```

```
GNIMMARGORP
GNIMMARGORP
NOHTYP
NOHTYP
MMING
TYP
GNIMMARGORP NOHTYP
```

Palindrome Check

```
name = "nayan"
if name == name[::-1]:
    print("Palindrom")
else:
    print("not palindrom")
```

```
Palindrom
```

String Iteration & Operations

Iterating Through Strings

```
s = "hello"

# Method 1: Direct iteration
for i in s:
    print(i)

# Method 2: Using range and indexing
for i in range(len(s)):
    print(i, s[i])
```

```
h
e
l
l
o

0 h
1 e
2 l
3 l
4 o
```

String Concatenation & Repetition

```
# Concatenation
s1 = "wel"
s2 = "come"
s3 = s1 + s2
print(s3)

# Adding space
```

```
s3 = s1 + " " + s2
print(s3)

# Repetition
s4 = s * 5
print(s4)

# Trying with float (will cause error)
s4 = s * 2.5
```

```
welcome
wel come
hellohellohellohellohello

TypeError: can't multiply sequence by non-int of type 'float'
```

Practical Applications

Display Cafe Menu

```
item1, price1 = "Tea", "10"
item2, price2 = "Coffee", "50"
item3, price3 = "Bournvita", "70"
item4, price4 = "Freshlime", "30"

print(f"{'MENU':^20}")
print(item1 + ("." * (20 - len(item1) - len(price1))) +
price1)
print(item2 + ("." * (20 - len(item2) - len(price2))) +
price2)
print(item3 + ("." * (20 - len(item3) - len(price3))) +
price3)
```

```
print(item4 + ("." * (20 - len(item4) - len(price4))) +
price4)
```

```
        MENU
Tea................10
Coffee.............50
Bournvita.........70
Freshlime.........30
```

Display Credit Card Number

```
card_no = input("Enter your credit card number:")
print((("X"*4) + " ") * 3 + card_no[-4:])
```

```
Enter your credit card number: 1111 2222 3333 4444
XXXX XXXX XXXX 4444
```

String Methods

Searching Methods

```
s = "Hello how are you"
print(s.find("o"))
print(s.find("o", 5))
print(s.find("o", 0, 4))
print(s.rfind("o"))
print(s.index("o"))
print(s.index("z"))
```

```
4
7
```

```
-1
15
4
ValueError: substring not found
```

> 💡 ***find()*** *returns -1 if not found, while* ***index()*** *raises ValueError*

Formatting Methods

```
s = "Hello"
print(s.ljust(10))
print(s.ljust(10, "_"))
print(s.rjust(10, "."))
print(s.center(10, "*"))

s = "123"
print(s.zfill(8))
```

```
Hello
Hello_____
.....Hello
**Hello***

00000123
```

Trimming Methods

```
s = " Hello "
print(s.lstrip())
print(s.rstrip())
print(s.strip())

s = "|_%@ _Hello|_%@ _"
print(s.strip("|_@ %"))
```

```
Hello
      Hello
Hello

Hello
```

Replacing & Splitting

```
s = "a-b-c-d-e"
print(s.replace("-", "/"))
print(s.replace("-", " "))
print(s.replace("-", ""))

s = "hi hello how are you?"
li = s.split()
print(li)
print(type(li))

s = "hi, hello, how, are, you"
li = s.split(",", 2)
print(li)
```

```
a/b/c/d/e
a b c d e
abcde

['hi', 'hello', 'how', 'are', 'you?']
<class 'list'>

['hi', ' hello', ' how, are, you']
```

Membership & Practical Problems

Membership Operators

```
s = "hello"
print("o" in s)
print("o" not in s)

li = [10, 8+9j, "hi", 5.3]
print(10 in li)
print("o" not in li)
```

```
True
False

True
True
```

Password Matching

```
pass1 = input("Enter your password:")
pass2 = input("Confirm your password:")
if pass1 == pass2:
    print("Passwords matched.")
elif pass1.lower() == pass2.lower():
    print("Check the case of your passwords.")
else:
    print("Passwords mismatched.")
```

```
Enter your password: MyPass123
Confirm your password: mypass123
Check the case of your passwords.
```

Email Processing

```
s = " Student.Name@GMAIL.com "
s = s.strip()
```

```
pos1 = s.find("@")
pos2 = s.rfind(".")
print(f"Username : {s[:pos1]}")
print(f"Domain Name: {s[pos1+1:].lower()}")
```

```
Username : Student.Name
Domain Name: gmail.com
```

Python String Operations - Lecture Notes #2

These notes cover essential string manipulation techniques in Python